

What is the Science of Programming

- The Science of Programming is about **program verification**. Program verification aims to get *reliable* programs by discerning properties about programs. In other words, it is about *reasoning* about correctness of programs as we write them: for each piece of code we write, we make sure that it gives us what we want.
 - For example, when you need create a while loop: “scan the array until you find a number that is no larger than 5”. This sounds easy to implement; but in real life it can be confusing: “Should I use ≤ 5 or < 5 or > 5 in the loop condition?” “Should I use $\&\&$ or $\|\|$ to connect the conditions?” Program verification can help you reason it out and create the correct loop.
- For this course, we'll look at a simple programming language.
 - The *syntax* will be simple because that's not the important part. What's important is formally (mathematically, logically) specifying the *semantics* of programs and connecting them to the semantics of logical statements.
 - As an aside: syntax means something related to grammar and semantics means something related to meaning.
- Test/debug vs Verification.
 - Testing + debugging can be extremely time consuming.
 - Testing cannot show correctness unless we have cases covering all possible situations; this might be a very large set. So, we reason our programs to identify a practical number of test cases that should represent all the possible test cases.
 - On the other hand, we can overlook cases while verifying programs (because the rules we use can be too strict). We need testing as a reality check to show that our reasoning is sound.
 - People usually combine these two techniques:
 - While testing, we usually look at statements in a program one by one; and we reason what result we expect after each statement.
 - While creating a test plan, we use logic to determine the test cases.
- 1. We have one statement in a program: “add integer x to z , if x is positive” and we have that variable $z \geq c$ before this statement, where c is a constant. What can be expected of variable z after this statement? How to decide test cases here?
 - We can write in this program in pseudo-code or any programming language easily, and a possible postcondition is given.

```
# z >= c
if x > 0:
    z += x
# z >= c
```

- We can have infinite numbers of test cases since there are infinite numbers of integers.

We can reason about how the statements and properties interact: take $x > 0$ (whose negation is $x \leq 0$) and break up $\leq$ into separate $<$ and $=$ cases ($x < 0, x = 0$), to get $x > 0, x = 0$, and $x < 0$ as the general

set of cases. If we think $x = 1$ and $x = -1$ are good enough generalizations of $x > 0$ and $x < 0$, then we're done: Our test cases are $x = -1, x = 0, x = 1$.

Of course, we can think of $x = 1$ and $x > 1$ as two different sub – cases for the case $x > 0$ (similarly $x = -1$ and $x < -1$); we can use $x = 2$ and $x = -2$ to test the cases $x > 1$ and $x < -1$ and we end up with five test cases: $x = -2, x = -1, x = 0, x = 1, x = 2$. If we keep breaking the cases, we will get an infinite number of cases; a big part of testing is figuring out when to stop doing this.

- As an aside, in this class we will prove that $\{z \geq c\}$ **if** $x > 0$ **then** $z := z + x$ **fi** $\{z \geq c\}$ is always valid so we don't need to test this piece of code at all.

Review of Propositional Logic

- **Logic** is the study of formal or valid reasoning. A **proposition** is a declarative sentence that is either true or false.
- **Propositional logic** is logic over **proposition variables** (usually expressed with letters $p, q, r \dots$), which are just variables that can have the values true or false.
 - In computer science terms, propositional logic is the logic used for Boolean expressions: *True* and *False* are Boolean constants.
- In propositional logic we study the **propositional operators**: and ($\wedge$), or ($\vee$), not ($\neg$), implication ($\rightarrow$), and biconditional ($\leftrightarrow$). Instead of discussing all the operators one by one, let's answer the following questions together.

2. Which of the following are propositions?

- | | |
|---|-------------------------------|
| a. IIT was founded in 1940. | Yes, and it is <i>False</i> . |
| b. Do you have any siblings? | No, it is a question. |
| c. Please log on to myIIT with your hawk credentials. | No, it is a command. |

3. With the propositions

p : It is below freezing. q : It is snowing.

Represent the following propositions using p, q , and logical connectives.

- | | |
|---|------------------------|
| (a) It is below freezing and snowing. | $p \wedge q$ |
| (b) It is below freezing but not snowing. | $p \wedge \neg q$ |
| (c) It is not below freezing, and it is not snowing. | $\neg p \wedge \neg q$ |
| (d) It is either snowing or below freezing (or both). | $p \vee q$ |

4. Translate each of the following to either $p \rightarrow q$ or $q \rightarrow p$:

- | | |
|------------------------------|-------------------|
| a. if p then q | $p \rightarrow q$ |
| b. p is sufficient for q | $p \rightarrow q$ |
| c. p if q | $q \rightarrow p$ |
| d. p is necessary for q | $q \rightarrow p$ |
| e. p only if q | $p \rightarrow q$ |

- The following propositions all means $p \rightarrow q$.

- "if p , then q "
- " p implies q "
- "if p, q "
- " p only if q "
- " p is sufficient for q "

- “a sufficient condition for q is p ”
 - “ q if p ”
 - “ q whenever p ”
 - “ q when p ”
 - “ q is necessary for p ”
 - “a necessary condition for p is q ”
 - “ q follows from p ”
 - “ q unless $\neg p$ ”
- $p \leftrightarrow q$ is the **biconditional** of p and q , it means $p \rightarrow q$ and $q \rightarrow p$ at the same time; in other words, $p \leftrightarrow q$ is *True* when p and q are both *True* or both *False*. But it doesn't mean p and q are “the same”.